

Setting up PREEMPT_RT

Building, installing and validating a real-time kernel on a Raspberry Pi 4 (4 GB, arm64)

docs/02_PREEMPT_RT_Setup_on_Raspberry_Pi_4.pdf | Course repository, document 2 of 2 | Seminar: Real-Time Linux — Fundamentals and Practical Applications | Trainer: Gaëtan Pandji | v1.5 · 22 August 2026

Prerequisite: Document 1 completed and `scripts/verify.sh` passing. Do not start here.
Scripts referenced in this document are in the repository — you do not need to retype the listings in section 12:

<code>scripts/build-rt-kernel.sh</code>	section 12.1 - run on your PC (cross-compile)
<code>scripts/switch-kernel.sh</code>	section 12.2 - run on the Pi
<code>scripts/verify.sh</code>	checks from document 1, section 7
<code>scripts/latency_showdown.sh</code>	day 1 demo: stock kernel vs. PREEMPT_RT
<code>config/rt-v8.config</code>	the exact <code>.config</code> this document produces
<code>DEFINITION_OF_DONE.md</code>	what "finished" means for each project stage

Clone once on the Pi: `git clone <repository-url> ~/rt-linux-seminar`

Starting point: a Raspberry Pi 4 running Raspberry Pi OS Lite (64-bit), Trixie, kernel 6.18.x, reachable over SSH — i.e. the end state of Document 1.

End state: two kernels installed side by side. The stock kernel stays bootable; a second, fully preemptible kernel boots on demand. `uname -a` shows PREEMPT_RT and `/sys/kernel/realtime` reads 1.

Time: 20–40 min cross-compiling on a PC, or 2–3 hours building natively on the Pi.

The single most important thing to know before you start. Since Linux **6.12**, PREEMPT_RT is **part of the mainline kernel**. There is no `patch-rt*.patch` to download and apply any more. Raspberry Pi OS now ships 6.18, so on this hardware the job is *configure and build*, not *patch, pray, and build*. Most tutorials you will find online predate this and will send you down an obsolete path.

1. What actually changes, and what you are choosing

Enabling `CONFIG_PREEMPT_RT` replaces most kernel spinlocks with priority-inheritance-aware sleeping mutexes, forces interrupt handlers into kernel threads, and breaks up long non-preemptible sections. The effect is a large reduction in *worst-case* latency, paid for with some throughput and a small constant overhead.

Preemption model	Behaviour	Use
PREEMPT_NONE	Server. Maximum throughput, worst latency.	Not here
PREEMPT / PREEMPT_LAZY	Desktop/embedded. What the stock Raspberry Pi OS kernel uses.	Your baseline for the comparison
PREEMPT_RT	Fully preemptible. Threaded IRQs, PI mutexes, bounded latency.	What you are building

Calibrate your expectations now. PREEMPT_RT *bounds* latency; it does not turn Linux into a microcontroller. An arm64 Pi 4 will not match a well-tuned x86 industrial PC, and neither matches FreeRTOS on bare metal for sub-microsecond determinism. Knowing where the ceiling is — and being able to defend that judgement in your own projects — is one of the intended outcomes of this seminar. We measure it on day 3 rather than asserting it.

2. Choose your build route

Route	Time	When to pick it
A — Cross-compile on an x86 Linux PC (or a VM / WSL2 / Docker)	10–30 min	Default choice. Fast, repeatable, and you can rebuild three times in an evening while you get the config right.
B — Native build on the Pi itself	2–3 h	When you have no Linux PC or VM available. Start it before a break or overnight — it will not finish inside a seminar slot. Run it under <code>tmux</code> so a dropped SSH session does not kill the build.

Both routes produce an identical kernel. Sections 3–5 give the commands for each; sections 6 onwards are common.

3. Prerequisites

Route A — on the build PC (Debian/Ubuntu)

```
sudo apt update
sudo apt install -y git bc bison flex libssl-dev make libc6-dev libncurses-dev \
    crossbuild-essential-arm64 rsync
```

Prove the toolchain resolves before you build anything. The kernel build constructs the compiler name by pasting `gcc` onto the end of `CROSS_COMPILE`. **The trailing hyphen is part of the value.**

```
export CROSS_COMPILE=aarch64-linux-gnu-      # correct - note the final hyphen
${CROSS_COMPILE}gcc --version                # MUST print a version string
```

Omit the hyphen and the build fails with C compiler 'aarch64-linux-gnugcc' not found — but only after several minutes of `kconfig` compilation, so it looks like a deeper problem than it is. See section 13.

Route B — on the Raspberry Pi

```
sudo apt update
sudo apt install -y git bc bison flex libssl-dev make libncurses-dev
```

On a native build, do not set `ARCH` or `CROSS_COMPILE` at all. If you previously followed Route A in the same shell, clear them first, or the native build will try to invoke a cross-compiler that is not installed:

```
unset ARCH CROSS_COMPILE
```

Check free space first: `df -h /`. The kernel tree plus objects needs roughly 12–15 GB. This is why Document 1 specifies a 32–64 GB card.

4. Get the kernel source

First, find out which branch matches the kernel you are running, so the device tree blobs and drivers stay consistent:

```
uname -r          # on the Pi, e.g. 6.18.34+rpt-rpi-v8 -> branch rpi-6.18.y
```

```
git clone --depth=1 --branch rpi-6.18.y https://github.com/raspberrypi/linux
cd linux
head -n 4 Makefile      # confirm VERSION / PATCHLEVEL / SUBLEVEL
```

`--depth=1` takes minutes instead of an hour. Omit `--branch` to get the current default branch. The full branch list is on github.com/raspberrypi/linux; if `rpi-6.18.y` no longer exists by the time you read this, pick the branch matching your `uname -r`.

5. Configure for real-time

5.1 Start from the board default configuration

```
# Route A (cross-compile), on the PC:
export ARCH=arm64
export CROSS_COMPILE=aarch64-linux-gnu-
KERNEL=kernel8
make bcm2711_defconfig
```

```
# Route B (native), on the Pi:
KERNEL=kernel8
make bcm2711_defconfig
```

`bcm2711_defconfig` is the correct 64-bit config for Pi 3, 3+, 4, 400, CM3/3+/4/4S and Zero 2 W. (Pi 5 would be `bcm2712_defconfig` with `KERNEL=kernel_2712`.)

5.2 The gotcha that stops most people

`PREEMPT_RT` is invisible in `menuconfig` until you enable `CONFIG_EXPERT`. In the kernel source, the option is declared as:

```
config PREEMPT_RT
    bool "Fully Preemptible Kernel (Real-Time)"
    depends on EXPERT && ARCH_SUPPORTS_RT && !COMPILE_TEST
```

`ARCH_SUPPORTS_RT` is selected by `arm64`, `x86` and `riscv` — so the `arm64` half of the dependency is already satisfied. `EXPERT` is not set in `bcm2711_defconfig`, which is why the menu entry simply is not there. People then conclude “the Pi kernel does not support RT”. It does.

5.3 Apply the configuration (scriptable, preferred)

```
./scripts/config --file .config --enable CONFIG_EXPERT
./scripts/config --file .config --disable CONFIG_PREEMPT_NONE
./scripts/config --file .config --disable CONFIG_PREEMPT_VOLUNTARY
./scripts/config --file .config --disable CONFIG_PREEMPT
./scripts/config --file .config --disable CONFIG_PREEMPT_LAZY
./scripts/config --file .config --disable CONFIG_PREEMPT_DYNAMIC
./scripts/config --file .config --enable CONFIG_PREEMPT_RT

# Drop the in-kernel debugger: unused here, costs latency, and breaks the link
./scripts/config --file .config --disable CONFIG_KGDB_KDB
./scripts/config --file .config --disable CONFIG_KGDB

# Name the kernel so it can never collide with the stock one
./scripts/config --file .config --set-str CONFIG_LOCALVERSION "-rt-v8"
./scripts/config --file .config --disable CONFIG_LOCALVERSION_AUTO

# Resolve dependencies, then PROVE it took effect
make olddefconfig
grep -E '^CONFIG_(EXPERT|PREEMPT_RT|PREEMPT_LAZY|LOCALVERSION)=' .config
```

Required output. You must see `CONFIG_PREEMPT_RT=y`. If you do not, nothing further in this document will work and you will have spent two hours building an ordinary kernel. Check `CONFIG_EXPERT=y` first, then open `make menuconfig` and read the dependency line with the **H** key.

5.4 Equivalent path through `menuconfig`

Alternative to 5.3, not a follow-on step. If the `grep` at the end of 5.3 printed `CONFIG_PREEMPT_RT=y`, your configuration is done — skip this section and go to section 6.

```
make menuconfig          # Route A: make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- menuconfig
```

1. **General setup** → enable *Configure standard kernel features (expert users)*
2. **General setup** → **Preemption Model** → select *Fully Preemptible Kernel (Real-Time)*
3. **General setup** → *Local version* → enter `-rt-v8`
4. Exit repeatedly, save when prompted

5.5 Optional determinism-related options

Option	Why you might want it
<code>CONFIG_NO_HZ_FULL=y</code>	Required if you want to use the <code>nohz_full=</code> boot parameter to stop the scheduler tick on an isolated core. Not enabled by default.
<code>CONFIG_HZ_1000=y</code>	1 kHz tick. Marginal under RT, but conventional for real-time configurations.
<code>CONFIG_CPU_FREQ_DEFAULT_GOV_PERFORMANCE=y</code>	Boots with the performance governor rather than ondemand. You can also do this at runtime (§9).
<code>CONFIG_VIRTUALIZATION=n</code> , <code>CONFIG_KVM=n</code>	Removes code paths you will never use on this board. Small, free win.
<code>CONFIG_DEBUG_PREEMPT=n</code> , <code>CONFIG_LATENCYTOP=n</code>	Debug instrumentation costs latency. Leave off unless you are debugging the kernel itself.
<code>CONFIG_KGDB_KDB=n</code> , <code>CONFIG_KGDB=n</code>	Recommended. The in-kernel debugger is not used anywhere in this course — we use <code>ftrace</code> , <code>perf</code> , <code>rtla</code> and userspace GDB instead — and it adds debug code that costs latency. It has also been observed to break the <code>vmlinux</code> link on this tree (section 13). Disabling it is the recommended default, not a workaround.

Change one thing at a time. Building a first RT kernel and simultaneously stripping thirty drivers is how you end up with a board that does not boot and no idea which change did it. Get a plain RT kernel booting, snapshot the `.config`, then tune.

6. Build

Check resources before starting a two-hour build. Running out of disk or memory during the module link stage is the most common late failure, and the error scrolls off screen before you see it.

```
df -h .          # needs ~25 GB free: tree + objects + several thousand .ko files
free -h         # rule of thumb: ~1.5 GB of RAM+swap per parallel job
```

With 4 GB and no swap, `-j4` can get the linker killed by the OOM killer. Lower `-j` rather than hoping.

Route A — cross-compile on the PC

```
make -j$(nproc) ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- \
  Image.gz modules dtbs 2>&1 | tee ~/build.log
```

Route B — native on the Pi

```
make -j6 Image.gz modules dtbs 2>&1 | tee ~/build.log
```

Always pipe through tee. A kernel build prints tens of thousands of lines; your terminal scrollback will not hold them. Without a log file, a failure 90 minutes in leaves you with only the final wrapper message, which tells you nothing. With one:

```
grep -n -i -E 'error|no space|cannot|not found|Killed' ~/build.log | head -40
```

Rule of thumb for `-j`: about $1.5 \times$ your core count. On a 4-core Pi 4 that is `-j6`. Watch the temperature with `vcgencmd measure_temp`; without a cooler this build will throttle, which is slow but harmless. Expect roughly 2–3 hours on a Pi 4.

7. Install — side by side, never on top

Design decision — do not shortcut this one. Do **not** overwrite `kernel8.img`. Install the RT kernel as a second file, `kernel8-rt.img`, and select it in `config.txt`. Three reasons: (1) the day-1 “Latency Showdown” comparison needs both kernels on the same board or the numbers prove nothing; (2) if your RT kernel fails to boot, you are one commented-out line away from a working system (section 11); (3) `apt full-upgrade` replaces the stock kernel and leaves yours untouched. This is the pattern used by `scripts/switch-kernel.sh`.

7.1 Back up the boot partition first

```
sudo tar czf ~/boot-firmware-backup-$(date +%F).tar.gz -C /boot/firmware .
```

7.2 Install the modules

```
# Route B - native, on the Pi:
sudo make -j6 modules_install

# Route A - cross-compile, card mounted on the PC at mnt/boot + mnt/root:
sudo env PATH=$PATH make -j$(nproc) ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- \
INSTALL_MOD_PATH=mnt/root modules_install
```

Because of `CONFIG_LOCALVERSION="-rt-v8"` the modules land in their own directory, e.g. `/lib/modules/6.18.34-rt-v8/`, and cannot clash with the stock kernel's modules.

7.3 Install the kernel image, DTBs and overlays

```
# Route B - native, on the Pi:
sudo cp arch/arm64/boot/Image.gz /boot/firmware/kernel8-rt.img
sudo cp arch/arm64/boot/dts/broadcom/*.dtb /boot/firmware/
sudo cp arch/arm64/boot/dts/overlays/*.dtb* /boot/firmware/overlays/
sudo cp arch/arm64/boot/dts/overlays/README /boot/firmware/overlays/
sync

# Route A - cross-compile, boot partition mounted at mnt/boot:
sudo cp arch/arm64/boot/Image.gz mnt/boot/kernel8-rt.img
sudo cp arch/arm64/boot/dts/broadcom/*.dtb mnt/boot/
sudo cp arch/arm64/boot/dts/overlays/*.dtb* mnt/boot/overlays/
sudo umount mnt/boot mnt/root
```

7.4 Select the RT kernel in `config.txt`

Edit `/boot/firmware/config.txt` and append:

```
[all]
kernel=kernel8-rt.img
auto_initramfs=0
```

Why `auto_initramfs=0`. Raspberry Pi OS builds its initramfs for the packaged kernel only; there is none for yours. `bcm2711_defconfig` compiles the SD/MMC controller and ext4 into the kernel, so it boots without an initramfs. **If your RT kernel hangs at boot**, generate a matching one instead:

```
sudo update-initramfs -c -k 6.18.34-rt-v8
sudo cp /boot/initrd.img-6.18.34-rt-v8 /boot/firmware/initramfs8-rt
# then in config.txt, replace auto_initramfs=0 with:
#   initramfs initramfs8-rt followkernel
```

```
sudo reboot
```

8. Verify — three checks, all of them

```

uname -a
# expect: Linux rtpi 6.18.45-rt-v8+ #1 SMP PREEMPT_RT ... aarch64 GNU/Linux
#               ^^^^^^^^^^^ this string is the point

# Confirm the string explicitly
uname -v | grep -o PREEMPT_RT
# expect: PREEMPT_RT      (empty output means wrong kernel)

lsmod | head           # modules load = modules_install went to the right place
dmesg | grep -i -E 'rt throttling|sched' | head

```

Note: `/sys/kernel/realtime` **may not exist on this tree**. That sysfs node is a convenience indicator created by some kernel versions, not by PREEMPT_RT itself. Its absence does not mean your kernel is wrong. The authoritative check is `uname -v | grep -o PREEMPT_RT` — the compiler embeds that string at build time when and only when `CONFIG_PREEMPT_RT=y`. Likewise, `/proc/config.gz` and `/boot/config-$(uname -r)` are absent on this tree; that is normal.

Definition of done for this step: `uname -v | grep -o PREEMPT_RT` prints `PREEMPT_RT`, the system boots to a login prompt, and `i2cdetect -y 1` still finds your sensor. That is sufficient. You do not need `/sys/kernel/realtime`.

9. Tune for determinism

An RT kernel alone buys you maybe half the improvement. The rest comes from getting other work off the core that matters.

9.1 Boot parameters

Edit `/boot/firmware/cmdline.txt`. It is **one single line** — do not introduce a newline, the Pi will not boot.

```
isolcpus=3 nohz_full=3 rcu_nocbs=3 irqaffinity=0-2
```

Parameter	Effect
<code>isolcpus=3</code>	The scheduler stops placing ordinary tasks on CPU 3. You put your RT thread there deliberately with <code>taskset</code> .
<code>nohz_full=3</code>	Stops the periodic scheduler tick on CPU 3 when one task is running. Requires <code>CONFIG_NO_HZ_FULL=y</code> (\$5.5) — silently ignored otherwise.
<code>rcu_nocbs=3</code>	Moves RCU callback processing off CPU 3.
<code>irqaffinity=0-2</code>	Routes interrupt handling to CPUs 0–2. This is the direct analogue of pinning housekeeping interrupts to core 0 while a real-time core runs undisturbed — the Schneider Electric case study in the course.

9.2 Clock and power management — the enemy of determinism

```

# Runtime: pin all cores to the performance governor
echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor

# Check what you actually got
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
vcgencmd measure_clock arm
vcgencmd get_throttled      # must stay 0x0 while measuring

```

To pin the ARM clock across reboots, add `force_turbo=1` to `config.txt`. This raises power draw and heat; it needs the active cooler and the proper 3 A supply. (Verify the current warranty-bit behaviour for your board revision before you recommend this to a customer — I have not re-checked it for 2026 firmware.)

9.3 Running a task as real-time

```

sudo chrt --fifo 80 taskset -c 3 ./my_rt_app      # priority 80, pinned to isolated core 3
chrt -p $(pidof my_rt_app)                       # confirm policy SCHED_FIFO

```

In code: `pthread_attr_setschedpolicy(SCHED_FIFO)` plus `mlockall(MCL_CURRENT|MCL_FUTURE)` to keep page faults out of your loop. Priorities 1–99; leave headroom above your task for kernel threads such as `irq/*` and `ktimersoftd`.

10. Measure — prove it, do not assert it

```
sudo apt install -y rt-tests stress-ng
```

10.1 The baseline / RT comparison ("Latency Showdown")

```
# Idle measurement, 5 minutes, with a histogram
sudo cyclicttest --mlockall --smp --priority=80 --interval=200 --distance=0 \
--duration=5m --histogram=400 --histfile=hist-idle.txt

# The one that actually matters: under load
sudo stress-ng --cpu 4 --io 2 --vm 2 --vm-bytes 256M --timeout 300s &
sudo cyclicttest --mlockall --smp --priority=80 --interval=200 --distance=0 \
--duration=5m --histogram=400 --histfile=hist-load.txt

# On the isolated core only
sudo cyclicttest --mlockall --priority=90 --interval=200 --affinity=3 --duration=5m
```

Read the **Max** column, not the average. Average latency is the number that flatters you; maximum latency is the number that decides whether a control loop misses a deadline. Then boot the stock kernel and repeat the identical command.

Do not quote numbers you have not measured on your own board — not mine, not a forum's. Card speed, PSU, cooling, firmware revision and load pattern all move the result. Run both kernels yourself, save both histograms, and show those. Commit your two histograms to `results/` in the repository — comparing them across machines is part of the day-3 discussion.

10.2 Modern alternative: `rtla`

`rtla timerlat` and `rtla osnoise` ship in the kernel tree and tell you not just *how much* latency you have but *where it came from*. Build them from the source tree you already cloned:

```
sudo apt install -y libtraceevent-dev libtracefs-dev pkg-config
cd ~/linux/tools/tracing/rtla && make && sudo make install

sudo rtla timerlat top -c 3 -d 60s      # timer latency on the isolated core
sudo rtla osnoise top -c 3 -d 60s      # what is stealing the CPU, broken down by source
```

These are a robust substitute if a Perfetto-based trace visualisation does not come together in time. Standard `ftrace`, `trace-cmd` and `perf` remain available and are the safest live-demo choice.

11. Recovery — when the Pi does not come back

It will happen at least once. This is a two-minute fix, not a re-flash.

1. Power off. Take the microSD card out and put it in a card reader on your PC.
2. The small FAT partition mounts as `bootfs`. Open `config.txt` in a plain text editor.
3. Comment out the kernel selection:

```
# kernel=kernel8-rt.img
```

4. Check `cmdline.txt` is still **exactly one line** and remove any `isolcpus/nohz_full` parameters you added.
5. Eject, boot. You are back on the stock kernel with your files intact.

This recovery path is the reason for the side-by-side install, and it is what makes deliberately breaking things a safe way to learn. **Practise it once on day 1, while nothing is wrong**. Knowing the fix takes two minutes is what lets you experiment freely on days 2 and 3 instead of tiptoeing around your own board.

12. Complete scripts

Both scripts are committed in the repository under `scripts/`. The listings below are for reading and for offline reference — run the repository copies, which are the authoritative versions and may have been fixed since this PDF was generated.

```
chmod +x scripts/*.sh
```

12.1 `scripts/build-rt-kernel.sh` — cross-compile, one command

```
#!/usr/bin/env bash
# Build a PREEMPT_RT kernel for Raspberry Pi 4 (arm64) on an x86 Linux host.
# Usage: ./build-rt-kernel.sh [branch]      e.g. ./build-rt-kernel.sh rpi-6.18.y
set -euo pipefail

BRANCH="${1:-rpi-6.18.y}"
SRC="$HOME/rpi-linux"
LOCALVER="-rt-v8"
JOBS="$(nproc)"
export ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-

echo "==> Dependencies"
sudo apt-get update -qq
sudo apt-get install -y git bc bison flex libssl-dev make libc6-dev \
    libncurses-dev crossbuild-essential-arm64 rsync

echo "==> Source (branch: $BRANCH)"
[ -d "$SRC" ] || git clone --depth=1 --branch "$BRANCH" \
    https://github.com/raspberrypi/linux "$SRC"
```

```

cd "$SRC"
head -n 4 Makefile

echo "==> Base configuration"
make bcm2711_defconfig

echo "==> Enabling PREEMPT_RT"
./scripts/config --file .config --enable CONFIG_EXPERT
for opt in PREEMPT_NONE PREEMPT_VOLUNTARY PREEMPT_PREEMPT_LAZY PREEMPT_DYNAMIC; do
./scripts/config --file .config --disable "CONFIG_$opt"
done
./scripts/config --file .config --enable CONFIG_PREEMPT_RT
./scripts/config --file .config --enable CONFIG_NO_HZ_FULL
./scripts/config --file .config --set-str CONFIG_LOCALVERSION "$LOCALVER"
./scripts/config --file .config --disable CONFIG_LOCALVERSION_AUTO
make olddefconfig

echo "==> Verifying configuration (build aborts here if RT is not set)"
grep -q '^CONFIG_PREEMPT_RT=y' .config \
|| { echo "FATAL: CONFIG_PREEMPT_RT is not set. Check CONFIG_EXPERT."; exit 1; }
grep -E '^CONFIG_(EXPERT|PREEMPT_RT|NO_HZ_FULL|LOCALVERSION)=_' .config

echo "==> Building with -j$JOBS"
time make -j"$JOBS" Image.gz modules dtbs

KREL="$(make -s kernelrelease)"
echo "==> Staging output for $KREL"
rm -rf "$SRC/out"; mkdir -p "$SRC/out/modules"
make -j"$JOBS" INSTALL_MOD_PATH="$SRC/out/modules" modules_install
cp arch/arm64/boot/Image.gz "$SRC/out/kernel8-rt.img"
mkdir -p "$SRC/out/dtb" "$SRC/out/overlays"
cp arch/arm64/boot/dts/broadcom/*.dtb "$SRC/out/dtb/"
cp arch/arm64/boot/dts/overlays/*.dtb* "$SRC/out/overlays/"

cat <<EOF

==> DONE. Kernel release: $KREL
Deploy to a Pi reachable at \${PI} (e.g. rt@rtpi.local):

rsync -a $SRC/out/modules/lib/modules/ \${PI}:/tmp/modules/
scp $SRC/out/kernel8-rt.img \${PI}:/tmp/
scp $SRC/out/overlays/* \${PI}:/tmp/overlays/
ssh \${PI} 'sudo tar czf ~/boot-backup.tar.gz -C /boot/firmware . && \
sudo cp -r /tmp/modules/* /lib/modules/ && \
sudo cp /tmp/kernel8-rt.img /boot/firmware/ && \
sudo cp /tmp/overlays/* /boot/firmware/overlays/ && sync'

Then enable it: sudo ./switch-kernel.sh rt && sudo reboot
EOF

```

12.2 scripts/switch-kernel.sh — run on the Pi

```

#!/usr/bin/env bash
# Switch between the stock and the PREEMPT_RT kernel, and toggle CPU isolation.
# Usage: sudo ./switch-kernel.sh {rt|stock|isolate on|isolate off|status}
set -euo pipefail
[ "$EUID" -eq 0 ] || { echo "Run with sudo."; exit 1; }

CFG=/boot/firmware/config.txt
CMD=/boot/firmware/cmdline.txt
RTIMG=kernel8-rt.img
ISO="isolcpus=3 nohz_full=3 rcu_nocbs=3 irqaffinity=0-2"

backup() { cp -n "$CFG" "$CFG.orig"; cp -n "$CMD" "$CMD.orig"; }

case "${1:-status}" in
rt)
[ -f "/boot/firmware/$RTIMG" ] || { echo "$RTIMG not found in /boot/firmware"; exit 1; }
backup
sed -i '/^kernel=/d; /^auto_initramfs=/d' "$CFG"
printf '\n[all]\nkernel=%s\nauto_initramfs=0\n' "$RTIMG" >> "$CFG"
echo "Selected: RT kernel. Reboot to apply."
;;
stock)
backup
sed -i '/^kernel=/d; /^auto_initramfs=/d' "$CFG"
printf '\nauto_initramfs=1\n' >> "$CFG"
echo "Selected: stock kernel. Reboot to apply."
;;
isolate)
backup
line="$(tr -d '\n' < "$CMD")"
for p in isolcpus nohz_full rcu_nocbs irqaffinity; do
line="$(echo "$line" | sed -E "s/[[:space:]]*${p}=[^[:space:]]*/g")"
done
if [ "${2:-}" = "on" ]; then line="$line $ISO"; fi
echo "$line" | tr -s ' ' > "$CMD" # MUST stay a single line
echo "CPU isolation: ${2:-off}. Reboot to apply."
echo "cmdline.txt is now:"; cat "$CMD"
;;
*)
echo "Unknown option: $1"
exit 1
;;

```

```
;;
status)
echo "Running kernel : $(uname -r)"
echo "Preemption      : $(uname -v)"
echo "Realtime flag    : $(cat /sys/kernel/realtime 2>/dev/null || echo 'not an RT kernel')"
echo "Selected image   : $(grep -E '^kernel=' "$CFG" || echo '(stock default)')"
echo "Boot cmdline      : $(cat /proc/cmdline)"
echo "Isolated CPUs     : $(cat /sys/devices/system/cpu/isolated)"
echo "Governor          : $(cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor)"
echo "Throttling        : $(vcgencmd get_throttled)"
;;
*) echo "Usage: sudo $0 {rt|stock|isolate on|isolate off|status}"; exit 1 ;;
esac
```

13. Troubleshooting

Symptom	Cause and fix
undefined reference to <code>`kdb_cmds'</code> at the LD <code>.tmp_vmlinux1</code> stage, with dangerous relocation messages	The in-kernel debugger (<code>CONFIG_KGDB_KDB</code>) failed to link. <code>kdb_cmds</code> comes from a generated source file; when that generation does not yield a linkable object, <code>kdb_main.o</code> references a symbol nothing defines. The dangerous relocation lines are secondary complaints about the same missing symbol. Fix: disable it — nothing in this course uses KDB. <pre>./scripts/config --file .config --disable CONFIG_KGDB_KDB ./scripts/config --file .config --disable CONFIG_KGDB make olddefconfig # re-run the same make command; the build resumes, it does not restart</pre>
make: *** [Makefile:248: __sub-make] Error 2 after modules appeared to link fine	This line is the wrapper reporting a failure below it, not the cause. The real error is 5–50 lines further up and has scrolled off. Re-run piping through <code>tee ~/build.log</code>, then <code>grep -n -i -E 'error no space cannot Killed' ~/build.log</code>. If output is still hard to read, re-run with <code>-j1</code> so the error appears directly above the failure.
Build dies during LD [M] ...	Almost always disk full — the module link stage writes several thousand <code>.ko</code> files. <code>df -h .</code>; you need roughly 25 GB. Free space or move the tree, then <code>make</code> again (the build resumes, it does not restart from scratch).
Build stops abruptly, Killed in the log	Out of memory. Confirm with <code>dmesg grep -i oom</code>. Reduce parallelism (<code>-j2</code> or <code>-j1</code>) or add swap. Budget roughly 1.5 GB of RAM+swap per job.
Link errors mentioning <code>elf</code> , <code>pahole</code> , BTF or <code>zstd</code>	Missing late-stage build dependency: <code>sudo apt install -y libelf-dev dwarves zstd</code>, then re-run <code>make</code>.
C compiler <code>'aarch64-linux-gnugcc'</code> not found followed by Error 1 on <code>bcm2711_defconfig</code>	Missing trailing hyphen in <code>CROSS_COMPILE</code>. The name is built as <code>\${CROSS_COMPILE}gcc</code>, so the value must end in <code>-</code>. Fix: <code>make mrproper</code>, then <code>export CROSS_COMPILE=aarch64-linux-gnu-</code>, then <code>\${CROSS_COMPILE}gcc --version</code> to confirm before rebuilding. If that still fails, install the toolchain: <code>sudo apt install crossbuild-essential-arm64</code>.
Same error, but you are building <i>on the Pi</i>	You exported the Route A cross-compile variables into a native build. <code>unset ARCH CROSS_COMPILE</code>, then <code>make mrproper</code> and start section 5.1 again on the Route B line.
<code>aarch64-linux-gnu-gcc: command not found</code>	Toolchain not installed on the build host: <code>sudo apt install -y crossbuild-essential-arm64</code>. Verify with <code>which aarch64-linux-gnu-gcc</code>.
<code>CONFIG_PREEMPT_RT</code> missing from <code>menuconfig</code>	<code>CONFIG_EXPERT</code> is off. See §5.2. This is the number-one cause.
Built fine, but <code>uname</code> shows plain <code>PREEMPT</code>	Either the config change did not stick (re-run the <code>grep</code> in §5.3), or the firmware booted the stock <code>kernel8.img</code>. Check the <code>kernel=</code> line in <code>config.txt</code>.
Kernel panic: VFS: Unable to mount root fs	Initramfs mismatch or missing modules. See the <code>auto_initramfs</code> box in §7.4; recover via §11.
Boots, but no network / no I ² C	<code>modules_install</code> did not reach <code>/lib/modules/\$(uname -r)</code>, or overlays were not copied. Check <code>ls /lib/modules/</code> and <code>dmesg grep -i firmware</code>.
Pi does not boot at all after editing <code>cmdline.txt</code>	You introduced a line break. It must be one single line. Fix it in a card reader.
<code>cyclictst</code> maxima worse than the stock kernel	Almost always thermal or power: check <code>vcgencmd get_throttled</code> and the governor. Then check that your isolated core is actually isolated: <code>cat /sys/devices/system/cpu/isolated</code>.
Cannot build out-of-tree modules	<code>/lib/modules/<ver>/build</code> does not point anywhere useful for a custom kernel. Run <code>make modules_prepare</code> in the source tree and symlink it.
<code>apt full-upgrade</code> seems to have undone things	It replaced the stock <code>kernel8.img</code> only. Your <code>kernel8-rt.img</code> and its modules are untouched. If <code>config.txt</code> was rewritten, re-run <code>switch-kernel.sh rt</code>.

14. Scope, assumptions and known limits

What is verified, what is assumed, and what is deliberately left out. Read this before you rely on the document in your own project.

- **Verified fact (19 Aug 2026):** PREEMPT_RT has been in mainline Linux since 6.12; CONFIG_PREEMPT_RT is declared with `depends on EXPERT && ARCH_SUPPORTS_RT && !COMPILE_TEST`; arm64 selects ARCH_SUPPORTS_RT; Raspberry Pi OS build 2026-06-18 ships Linux 6.18.34. The build, install and `config.txt` steps follow the official Raspberry Pi kernel documentation.
- **Why that matters for your own research:** the large majority of PREEMPT_RT tutorials online predate September 2024 and instruct you to download and apply an `rt` patch. On 6.12 and later that step no longer exists. If a guide tells you to `wget patch-*.rt*.patch`, it is describing an older kernel.
- **Assumption:** boot from microSD using `bcm2711_defconfig`, whose SD/MMC and ext4 drivers are compiled in — that is what makes `auto_initramfs=0` viable in section 7.4. Booting from USB or NVMe changes this; use the `initramfs` fallback.
- **Open items, not verified on hardware at the time of writing:** (a) the exact interaction between `auto_initramfs` and a custom `kernel=` filename on 6.18 firmware; (b) the current `force_turbo` warranty-bit behaviour on Pi 4. Both are flagged inline where they appear. If you resolve either on your board, please report it — it improves the document for the next cohort.
- **Deliberately absent: latency numbers.** No figure is printed anywhere in this document. Card speed, power supply, cooling, firmware revision and load pattern all move the result by more than the differences people argue about online. You will measure your own numbers in section 10, and those are the only ones you should ever quote.
- **Reasoning behind the workflow:** build once, cross-compiled, from a script, with the resulting `.config` committed to version control. A kernel you cannot rebuild from a script is not an asset — it is a card you are afraid to reformat. That principle is the reason this repository exists in the form it does, and it transfers directly to production embedded work.

15. Contributing and change log

If a command here fails on your hardware, that is a defect in the document. Open a repository issue with the command, the full output, `uname -a`, and your image build date. Fixes to `scripts/` are welcome as pull requests — the branch-naming and pull-request conventions are in `README.md`, and using them is itself part of the course.

Version	Date	Change
v1.5	22 Aug 2026	Section 8: removed <code>/sys/kernel/realtime</code> as a required check. On the Raspberry Pi 6.18 tree that file does not exist; its absence does not indicate a problem. Authoritative check is <code>uname -v grep -o PREEMPT_RT</code> . Note added explaining why <code>/proc/config.gz</code> and <code>/boot/config-\$(uname -r)</code> are also absent.
v1.4	21 Aug 2026	Sections 5.3 and 5.5 now disable <code>CONFIG_KGDB_KDB</code> / <code>CONFIG_KGDB</code> by default, after a reported <code>undefined reference to kdb_cmds</code> failure at the vmlinux link. Section 13 documents the symptom and fix. <code>build-rt-kernel.sh</code> updated to match.
v1.3	21 Aug 2026	Section 6 gained a disk and memory preflight and now pipes both routes through <code>tee</code> . Four build-failure entries added to section 13, including how to read <code>__sub-make</code> Error 2. <code>build-rt-kernel.sh</code> checks resources before building and greps its own log on failure.
v1.2	20 Aug 2026	Pagination fix: lists no longer split across pages. Section 5.4 now states explicitly that it is an alternative to 5.3, not a subsequent step.
v1.1	20 Aug 2026	Added toolchain preflight check to section 3 and three cross-compiler troubleshooting entries to section 13, after a reported <code>aarch64-linux-gnugcc not found</code> failure caused by a missing trailing hyphen in <code>CROSS_COMPILE</code> . <code>build-rt-kernel.sh</code> now validates the toolchain before starting.
v1.0	19 Aug 2026	Initial release. Targets Raspberry Pi OS 2026-06-18 / Linux 6.18.34 on Raspberry Pi 4 (BCM2711, arm64).

Previous: [docs/01_Raspberry_Pi_OS_Download_and_Flash.pdf](#) · Repository index: [README.md](#) · Acceptance criteria: [DEFINITION_OF_DONE.md](#)
Kernel source: [github.com/raspberrypi/linux](#) · Official build docs: [raspberrypi.com/documentation/computers/linux_kernel.html](#)